Deep Learning

Elisa Ricci
a.y. 2025/2026
University of Trento

Davide Donà

February 2026

Notes Repository

Contents

1 Introduction **1**

1.1 Basic Definitions 1

1.2 Historical Background 1

1 Introduction

In this section we will briefly introduce the basic concepts of deep learning, its historical background, and a brief recap of machine learning concepts that are fundamental to understanding deep learning.

1.1 Basic Definitions

AI Artificial Intelligence (AI) is a discipline that has the ambition to create machines that mimic human intelligence. It encompasses a wide range of techniques and approaches, including rule-based systems, expert systems, and machine learning. The ultimate goal of AI is to make programs that can learn from their experience as effectively as humans do.

Machine Learning Machine Learning (ML) is a subset of AI that uses techniques that enable machines to learn from experience (i.e. data) to improve their performance on a specific task.

Deep Learning is an area of Machine Learning that in particular focuses on `Artificial Neural Networks` (ANNs). The term "deep" refers to the use of multiple layers in the network, which allows it to learn complex representations of data, with multiple levels of abstraction.

1.2 Historical Background

The concept of artificial neural networks dates back to the 1940s, with the work of Warren McCulloch and Walter Pitts, who proposed a simple model that tried to emulate the behavior of biological neurons. This can be seen as the first implementation of a single layer perceptron, which is a fundamental building block of deep learning.

Electronic Brain In the first perceptron model, the input is a vector of **binary features** and all the inputs are simply summed together and passed through a **treshold function**, without any **weights** involved.

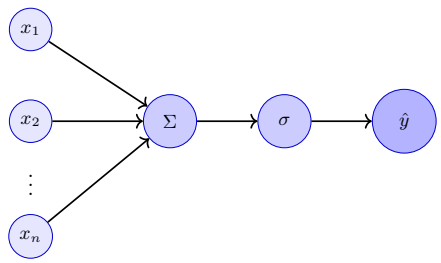

Figure 1: Architecture of the electronic brain model. Inputs x_i are summed together and passed through an activation function σ to produce the output $\hat{y}$.

It can be shown that this model is capable of classifying linearly separable data, such as the AND, OR and NOT functions, but it fails to classify non-linearly separable data, such as the XOR function.

The Perceptron In the 1950s, Frank Rosenblatt introduced the concept of the **Perceptron**, which is an extension of the electronic brain model. The difference is found within the introduction of **weights** for each input, which allows the model to learn from data and adjust its parameters accordingly.

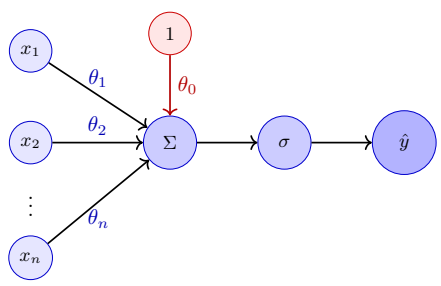

Figure 2: Architecture of a single layer perceptron. Inputs x_i are weighted by w_i, summed together, and passed through an activation function σ to produce the output y.

The prediction $\hat{y}$ of the perceptron is given by the following formula:

$$\hat{y} = \sigma\left(\sum_{i=1}^{n} \theta_i x_i\right) \tag{1.1}$$

where θ_i are the weights associated with each input x_i, and σ is the activation function. We can also express the prediction in vector form as follows:

$$\hat{y} = \sigma\left(\boldsymbol{\theta}^T \boldsymbol{x}\right) \tag{1.2}$$

The most important intuition is that the weights θ_i are not fixed, but rather learned from data.

The notion of bias is also introduced, which allows the model to shift the decision boundary (hyperplane) away from the origin.

Perceptron Learning Algorithm The weights of the perceptron are learned from data using the **Perceptron Training Rule**. Given a set of training examples $\mathcal{T} = \{(\boldsymbol{x_1}, y_1), (\boldsymbol{x_2}, y_2), \ldots, (\boldsymbol{x_N}, y_N)\}$, $y_i \in \{0, 1\}$ and a learning rate η, the weights are updated as follows:

1. Initialize the weights θ_i to small random values;
2. Until convergence, repeat:
   (a) Select randomly a training example $(\boldsymbol{x_i}, y_i)$ from $\mathcal{T}$;
   (b) Compute the prediction $\hat{y}_i$ using the current weights:
   $$\hat{y}_i = \sigma\left(\theta^T \boldsymbol{x_i} + \theta_0\right)$$
   (c) Update the weights based on the error between the prediction and the true label:
   $$\theta^{(t+1)} \leftarrow \theta^{(t)} + \eta(y_i - \hat{y}_i)\boldsymbol{x_i}$$
   (d) Update the bias:
   $$\theta_0^{(t+1)} \leftarrow \theta_0^{(t)} + \eta(y_i - \hat{y}_i)$$

Note that we are assuming that the training data is linearly separable, otherwise we would have to introduce a stopping criterion to prevent the algorithm from running indefinitely.

Percepton Today Today, the activation function σ is typically a non-linear function, such as the sigmoid, ReLU, or tanh function, which allows the model to learn complex patterns in the data.

Perceptron

An artificial neuron, i.e. a **percepton** is a non-linear parametrized function with a restricted output range.

Perceptron Problem As anticipated, in the 1970s it was found that the perceptron was not able to learn non-linearly separable data, such as the XOR function. This was seen as a major setback for the field of AI, and it led to a decline in interest and funding for research in this area, which is now known as the "AI winter".

MLP AI did not stop there, and in the 1980s, researchers introduced the concept of **Multilayer Perceptron** (MLP). This architecture had a major issue, which was the difficulty in computing the weights of the hidden layers.

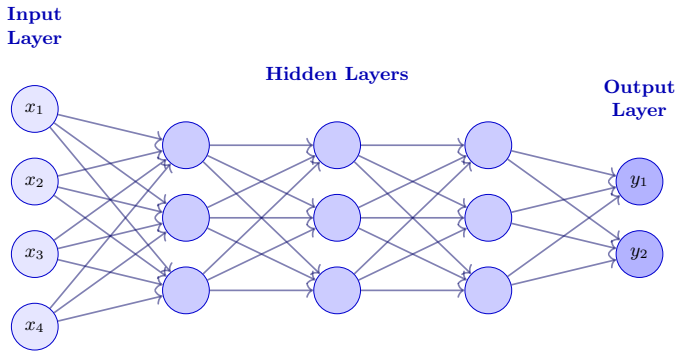

Figure 3: Architecture of a Multi-Layer Perceptron (MLP) with three hidden layers, fully connected in a feed-forward fashion.

This structure was very flexible as multiple nodes could be used in the output layer, so it could be used for multi-class classification problems, and also for regression.

Backpropagation The perceptron learning algorithm was not suitable for training MLPs, as no expected output was available for the hidden layers, but just for the output layer.

Backpropagation is an efficient algorithm for computing the gradients also for large datasets. At a high level, the algorithm can be summarized as follows:

1. **Forward propagation**: Given an input $\boldsymbol{x}$, compute the output of the network $\hat{y}$ by propagating the input through the layers of the network;
2. **Loss computation**: Compute the loss $L(y, f(\boldsymbol{x}, \boldsymbol{\theta}))$, where y is the true label and $f(\boldsymbol{x}, \boldsymbol{\theta})$ is the output of the network with parameters $\boldsymbol{\theta}$;
3. **Backward propagation**: the error signal is propagated backwards through the network and it's used to update the weights.

Obviosly, this concept will be covered in more detail in the next chapters.

Other Architectures Other architectures were proposed in the following years that lead to the current breakthrough of deep learning, such as:

- **Convolutional Neural Networks** (CNNs), which are particularly effective for image processing tasks;

- **Recurrent Neural Networks** (RNNs), used for sequential data, such as time series and natural language processing;

Cybenko's Theorem Cybenko's theorem states that a 2-layer feed-forward neural network with linear output can approximate any continuous function on a compact subset of $\mathbb{R}^n$ with arbitrary accuracy.

In practice, the more hidden layers we have, the more complex functions we can learn. Deep neural networks need a lot of labeled data to be trained, and cannot be trained easily.

Second Winter of AI Three main factors contributed to the second AI winter in the late 1980s:

- **Deep Neural Networks** tend to overfit the training data, and had problems with vanishing gradients, which made them difficult to train;
- **Lack of computational resources** to train deep neural networks. In fact, no GPUs were available at the time;
- **Lack of large datasets** to train deep neural networks.

All this issues led to a decline in interest and funding for research in this area, with an increase of interest in other areas of ML, such as Support Vector Machines (SVMs) and Kernel Methods.

First Deep Learning Applications - AlexNet In 2012, a deep convolutional neural network called AlexNet won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with a top-5 error rate of 15.3%, which was significantly better than the second-best entry, which had an error rate of 26.2%. This was a major breakthrough for the field of deep learning, and it led to a shift of interest and funding back to this area, which is now known as the "Deep Learning Revolution".

AlexNet also shifted the focus of research from shallow architectures to deep architectures. The first one required heavy works on feature engineering before training the model, while the second one was able to learn features directly from the data, without any human intervention.